

Slam 课程项目

3DGS-based SLAM 复现

报告

学号: 2353924

姓名: 冯俊财

日期: 2026-6-24

目录

1. 引言	1
2. 个人负责内容与分工	1
3. 实验环境与数据集说明	2
3.1. 实验环境	2
3.2. 数据集说明	2
4. VINGS-Mono 复现、遇到的问题及处理	3
4.1. 复现准备与整体实验步骤	3
4.2. KITTI 数据集复现过程与配置调整	3
4.3. KITTI360 长序列复现与稳定性优化	4
4.4. VIO 相关问题排查与兼容修复	5
4.5. 渲染结果、轨迹输出与运行状态判断	5
4.6. 输出组织、checkpoint 与中间结果管理	6
4.7. 移动端专用配置的前期准备	6
5. 移动端实时 VO 服务复现、遇到的问题及处理	6
5.1. 移动端目标与整体链路设计	6
5.2. 服务端运行链路搭建	7
5.3. 客户端采集方式与参数适配	7
5.4. 通信链路与时延性问题处理	8
5.5. 图像方向、回传布局与结果解释	8
5.6. 连接管理与客户端稳定性改进	9
5.7. 为什么最终固定为 VO 模式而未继续推进 VIO	9
6. 实验结果与分析	10
6.1. 离线数据集复现结果	11
6.2. 长序列与大场景结果分析	12
6.3. 移动端实时 VO 联调结果	12
6.4. 综合分析	13
7. 总结	14

1. 引言

随着机器人感知、三维重建和场景理解等方向的发展，如何在真实环境中实现稳定的定位与高质量建图，成为视觉 SLAM 领域的重要研究问题。传统视觉 SLAM 方法通常侧重于相机位姿估计和稀疏地图构建，虽然在定位方面已经取得了较好的效果，但在场景细节表达、稠密重建质量以及新视角渲染能力方面仍存在一定局限。近年来，Gaussian Splatting 作为一种高质量、可微分且渲染效率较高的三维场景表示方法，逐渐被引入 SLAM 系统中，为同时实现定位、建图与渲染提供了新的技术路径。

VINGS-Mono 是一套面向大规模场景的单目视觉惯性 Gaussian Splatting SLAM 框架，主要用于解决大尺度场景下单目建图容易出现的漂移、回环误差累积、动态物体干扰以及大规模 Gaussian 地图维护等问题。根据论文介绍，该系统主要由视觉惯性前端、2D Gaussian 地图模块、基于新视角合成的回环检测模块以及动态物体消除模块构成，能够在仅使用相机和低频 IMU 的条件下，实现较稳定的位姿估计与高质量稠密场景重建。与已有一些主要面向小范围室内环境的 GS-SLAM 方法相比，VINGS-Mono 更强调对室外大场景和长轨迹任务的支持，并在驾驶场景、无人机场景和室内场景中展示了较好的效果。

2. 个人负责内容与分工

在本次课程项目中，我个人主要负责两方面工作：一是 VINGS-Mono 在 VO / VIO 相关数据集上的复现、配置调整与运行调试；二是移动端实时 VO 功能的构建与联调。前一部分工作的重点在于围绕不同数据集的输入格式、配置参数和运行流程，对开源项目进行适配与验证，尽可能将论文方法落实到可执行的实验链路中；后一部分工作的重点在于面向移动端实时使用场景，搭建客户端与服务端之间的通信与处理流程，并完成整体功能测试。

在 VO / VIO 数据集复现方面，我的主要工作包括以下几个方面。首先，我负责整理项目运行所需的实验环境，包括代码仓库、依赖项、预训练权重、数据目录结构以及服务器运行所需的基础配置。其次，我根据不同数据集的组织方式，对配置文件中的数据路径、相机内参、图像分辨率、IMU 相关参数和可视化选项进行调整，并结合脚本修改和多次实验，完成项目在不同场景下的启动与运行。再次，在复现过程中，我对运行日志、输出文件和中间结果进行检查，分析程序在轨迹估计、渲染、建图和显存占用等方面出现的问题，并通过参数调整、链路修补和稳定性优化，使系统能够较稳定地完成实验流程。

在移动端实时 VO 功能构建与联调方面，我主要负责服务端运行链路搭建、移动端与服务端通信流程调整，以及移动端构建兼容性处理。具体来说，我围绕移动端实时视觉里程计场景，配置了服务端运行脚本和对应的移动端专用配置文件，打通了手机端图像与 IMU 数据向服务端传输、服务端完成跟踪与建图、再将可视化结果回传到客户端的整体流程。在此基础上，我还对客户端连接逻辑、视频流发送方式、服务端数据接收与队列处理方式进行了调整，以提升实时运行的稳定性和联调成功率。此外，针对移动端构建过程中出现的 Flutter、Gradle 和 Android 构建环境兼容问题，我也进行了相应修复，使移动端程序能够顺利完成编译、部署和测试。

总体来看，我在本项目中的工作不属于算法设计层面的完整实现，而更偏向于工程复现、

系统调试、配置适配和联调验证。我的任务重点是将论文与开源项目提供的方法落实到具体实验条件下，尽可能跑通 VO / VIO 数据集复现流程，并完成移动端实时 VO 的基础搭建与验证，为小组项目的实验实现和结果分析提供可执行的系统支撑。

3. 实验环境与数据集说明

3.1. 实验环境

本次实验主要采用“本地开发调试 + 远端服务器运行 + 移动端联调验证”的方式完成。其中，本地环境主要用于代码管理、配置修改、脚本编写、日志整理以及移动端应用构建；远端服务器主要用于运行 VINGS-Mono 的跟踪、建图与渲染流程；移动端设备主要用于实时采集图像并完成与服务端的联调测试。

在本地开发环境方面，主要使用 Windows 11 作为开发平台，完成代码仓库管理、配置文件修改、远程连接以及 Android 客户端构建等工作。移动端客户端基于 Flutter 框架进行构建和调试，相关构建过程中涉及 Android SDK、Gradle 和 Flutter 版本兼容问题，因此本地环境同时承担了移动端工程适配与编译验证的任务。

在远端运行环境方面，VINGS-Mono 主要部署在 Linux GPU 服务器上运行。根据项目环境说明和实际运行记录，实验使用的 Python 版本为 3.9.19，CUDA 版本为 11.8，深度学习框架以 PyTorch 2.0.1 + cu118 为主。由于 VINGS-Mono 同时包含前端跟踪、后端 Gaussian 建图和渲染相关过程，对 GPU 显存和运行稳定性有较高要求，因此实验主要依赖带 24GB 显存的 GPU 环境的远端服务器，来运行长序列实验和移动端实时链路。

除基础运行环境外，项目还依赖若干预训练模型与第三方模块。例如，前端跟踪过程需要使用 droid.pth 权重文件，回环或特征相关流程需要 LightGlue 对应模型文件，此外项目本身还包含与 2D Gaussian Splatting、深度估计及相关视觉处理有关的依赖模块。

对于移动端实时 VO 部分，实验环境由三部分组成：Android 手机客户端、本地中转设备以及远端 GPU 服务器。手机端负责实时采集图像与 IMU 数据；本地设备主要承担 SSH 隧道转发和客户端调试；远端服务器则负责运行 VINGS-Mono 的跟踪与建图流程，并将结果回传给客户端显示。这样的环境组织方式有助于降低移动端直接访问远端服务器时带来的网络不稳定因素，也便于在调试过程中分别定位客户端、网络链路和服务端的问题。

3.2. 数据集说明

涉及的数据主要分为两类：一类是用于 VO / VIO 复现与调试的公开数据集，另一类是用于移动端实时联调的手机端实时采集数据流。

在公开数据集复现部分，本次使用的是 KITTI 与 KITTI360 相关序列。

KITTI 数据集主要用于验证 VINGS-Mono 在驾驶场景中的视觉惯性运行流程。该数据集具有典型的室外道路场景特征，轨迹较长、运动连续，能够较好地反映系统在真实车辆运动环境下的定位与建图能力。KITTI360 数据集则进一步提高了场景规模和运行复杂度，更适合用于测试系统在大场景、长序列条件下的稳定性与资源占用情况。相较于 KITTI，KITTI360 对建图规模、渲染链路和显存管理提出了更高要求。

除公开数据集外，还有移动端实时 VO 链路的数据接入与联调验证。在这一部分中，

系统输入不再是离线整理好的图像序列，而是由 Android 手机客户端实时采集并发送到服务端的图像数据流。与此同时，移动端也会同步发送 IMU 数据，但结合当前联调结果来看，移动端链路中的 IMU 数据稳定性仍不足以支撑稳定的 VIO 运行，因此本次移动端实验最终以 VO 模式为主，即以实时图像输入为核心完成跟踪、建图和结果回传测试。

4. VINGS-Mono 复现、遇到的问题及处理

在完成实验环境准备和数据整理之后，后续工作进入到 VINGS-Mono 的实际复现、调试与稳定性优化阶段。由于该项目同时涉及前端视觉/视觉惯性跟踪、后端 Gaussian 建图与结果渲染，而且不同数据集在图像分辨率、数据组织方式、相机内参、IMU 频率和运行负载上存在明显差异，因此公开仓库中的默认配置并不能在当前实验条件下直接稳定运行。本阶段的主要任务，是围绕公开数据集复现链路，对配置文件、运行脚本和部分辅助逻辑进行适配与修正，使系统能够完成从数据读取、位姿估计、建图训练到结果输出的完整流程。

4.1. 复现准备与整体实验步骤

本阶段工作采用了“环境检查 - 配置适配 - 启动实验 - 日志分析 - 参数回调 - 结果整理”的迭代式流程，而不是一次性运行完成。具体而言，首先根据仓库说明准备运行环境，包括 Python 3.9、CUDA 11.8、PyTorch 运行条件以及前端跟踪所需的 droid.pth 权重、LightGlue 相关模型文件等。随后检查数据集目录结构是否与配置文件中的 dataset.root 一致，并确认输出目录与权重路径在服务器环境下可访问。

在环境准备完成后，主要围绕 YAML 配置文件进行适配。实验中重点修改或核对的配置项包括：dataset.root、frontend.weight、output.save_dir、mode、图像输入尺寸、相机内参、imu_delay、训练迭代次数、在线可视化选项以及存储管理参数。其中，dataset.root 决定数据是否能够被正确读取，frontend.weight 决定前端模型是否能够正常加载，mode 决定系统以 VO 还是 VIO 方式运行，而图像尺寸、相机内参与训练参数则直接影响跟踪稳定性与建图负载。

完成配置适配后，再通过对应脚本启动实验。离线公开数据集的主要入口为 scripts/run.py，移动端相关实验则使用独立入口。实验启动后，结合终端日志、中间生成文件和输出目录对运行状态进行判断。判断一组实验是否“跑通”，主要依据以下几个方面：其一，程序是否能够连续处理完整数据序列而不中途退出；其二，是否生成轨迹文件；其三，是否生成 rgbdnua 渲染结果；其四，是否能够输出最终 ply 地图模型。如果上述结果不完整，则继续回到配置文件或代码中进行修正。

从实验流程上看，公开仓库的复现并不是“直接运行脚本”即可完成，而是需要不断在配置、日志和结果之间来回检查。后续 KITTI、KITTI360 以及移动端部分的所有工作，基本都沿用了这一流程。

4.2. KITTI 数据集复现过程与配置调整

KITTI 相关实验主要用于验证 VINGS-Mono 在驾驶场景下的视觉惯性运行能力。本次复现中，重点使用了 KITTI 同步序列配置，例如 configs/kitti/sync/kitti_2011_09_30_drive_0028.yaml 和 configs/kitti/sync/kitti_2011_09_30_drive_0027.yaml。这些配置文件中已经给出

了数据读取模块、图像分辨率和相机参数，但由于默认路径和当前服务器环境不一致，因此首先需要完成路径适配。

具体步骤上，首先修改 `dataset.root`，使其指向服务器上的实际数据目录；随后检查 `frontend.weight` 是否指向当前环境中的 `droid.pth`，并确认 `output.save_dir` 指向可写输出路径。完成路径修正后，再检查与运行效果关系更强的参数，包括 `mode: 'vio'`、图像输入尺寸、相机内参和 `imu_delay`。其中，`imu_delay` 是 KITTI 复现中一个需要重点关注的参数。因为视觉与惯性信息并非天然严格同步，如果该参数设置不合理，系统虽然可以启动，但容易在初始化后出现轨迹异常、漂移增大甚至整体结果失稳。

在初步跑通配置后，还需要继续调整前端和训练参数，以适应当前计算资源和目标任务。例如，前端中的 `buffer`、`frontend_window`、`active_window`、`keyframe_thresh`、`frontend_thresh` 等参数会影响关键帧选择和位姿估计过程；训练参数中的 `iters`、学习率和各类损失项权重则会影响后端地图优化负载。

KITTI 复现过程中还遇到了与 VIO 运行链路相关的问题。由于视觉惯性部分依赖较强，且当前环境与原始开发环境并不完全一致，因此曾出现过与 GTSAM 相关的兼容性问题。这类问题的特点是：即使配置文件本身正确，系统也可能因为底层接口不一致而无法进入正常运行状态。针对这一情况，实验中对相关兼容层进行了补充处理，使前后端调用链能够在当前环境下继续执行。

从结果上看，经过以上调整后，KITTI 序列已经能够完成完整运行，并生成轨迹文件、渲染图像和地图模型。这表明系统在驾驶场景下的基本复现链路已经建立起来。虽然部分轨迹精度仍与理想结果存在差距，但从课程项目角度看，KITTI 部分已经完成了“能够实际运行并输出结果”的复现目标。

4.3. KITTI360 长序列复现与稳定性优化

相比 KITTI，KITTI360 的场景规模更大、图像分辨率更高、序列长度更长，因此更容易暴露系统在显存、渲染和长时间运行稳定性方面的问题。本次复现中，重点使用了 `configs/kitti360/unsync/kitti360_2013_05_28_drive_0002.yaml` 以及若干长序列/轻量化配置，用于测试系统在大场景条件下的可运行性。

这部分实验的第一步同样是配置适配。首先修改 `dataset.root`、`frontend.weight` 和输出目录，确保程序能够正确读取数据并保存结果。随后检查输入图像尺寸和相机内参是否与当前序列一致，并确认系统以 `mode: 'vio'` 或相应目标模式运行。由于 KITTI360 本身图像分辨率较高，如果直接使用较高负载的训练和可视化设置，系统在运行中很容易出现计算开销过大和显存占用持续上升的问题，因此还需要进一步对训练参数和可视化选项进行压缩。

在具体处理上，主要采取了“保留核心链路、压缩运行负载”的思路。一方面，保留前端跟踪、后端建图和结果输出所必需的主要模块；另一方面，适当降低训练迭代负载，并根据实验目的调整 `use_vis` 和存储管理相关配置。对于优先追求“长序列完整跑通”的实验，在线可视化不再作为首要目标，而是优先保证程序能够持续运行并保留最终结果。这样做的目的是在单卡 24GB 显存环境下提高实验完成率。

KITTI360 复现中最典型的问题之一是空渲染和渲染异常。在部分长序列实验中，系统

虽然没有立即退出，但中间渲染结果会出现空图或有效内容严重缺失。如果不及时处理，这种异常状态会继续传递到后端训练中，最终导致结果失真甚至运行中断。针对这一问题，实验中重点结合日志输出和中间图像进行排查，并在相关训练流程中补充了必要的异常保护逻辑，用于在渲染结果为空时跳过异常状态或阻止其继续扩散。这样做并不是改变系统主流程，而是为了避免单一异常帧直接导致整个长序列实验失败。

另一个突出问题是显存和中间状态增长过快。随着长序列推进，系统会持续积累新的 Gaussian 和相关状态信息，如果缺乏控制，GPU 端占用会不断增加，最终影响运行稳定性。因此，实验中还存储管理参数进行了重点调整，包括控制 GPU 端 Gaussian 数量上限、降低部分中间结果负载，以及利用定期管理策略减小资源压力。与此同时，还补充整理了 checkpoint 和中间结果保存策略，使实验即使在长时间运行后，也能留下轨迹、渲染结果和最终地图模型等关键输出。

在实际运行中，KITTI360 的复现不再是“能否启动”的问题，而是“能否完整跑完并留下可分析结果”的问题。经过多轮调试后，若干长序列实验最终能够生成 droid_c2w、rgbdnua 和最终 ply 等结果文件。这说明系统在当前条件下已经具备了基本的大场景长序列可运行性，也表明稳定性优化在这部分实验中发挥了关键作用。

4.4. VIO 相关问题排查与兼容修复

在公开数据集复现中，VIO 模式本应是 VINGS-Mono 的重要特点之一，但在实际运行中，其稳定性明显比纯视觉链路更依赖参数设置和底层环境。因此，VIO 相关问题排查本身构成了本阶段工作的重要内容。

首先需要关注的是配置层面的视觉惯性参数，包括运行模式、相机与 IMU 的对齐关系以及时间延迟设置。以 KITTI 为例，imu_delay 的设置是否合适，会直接影响系统是否能够完成稳定初始化。如果该参数明显偏离数据实际情况，即使前端图像读取和网络权重加载全部正常，系统仍可能在初始化后出现位姿异常、轨迹发散或整体不稳定。因此，在调试过程中，并不是只检查“脚本是否能运行”，而是要结合输出结果判断视觉和惯性信息是否真正形成了合理配合。

其次是依赖兼容问题。VIO 链路涉及 GTSAM 等底层依赖，而当前实验环境与原始仓库默认环境并不完全一致，因此曾出现过接口不兼容或导入链异常的问题。这类问题的特征是：配置文件表面正确，但程序在进入视觉惯性相关分支后无法正常执行。针对这一情况，实验中对相关兼容接口进行了补充处理，使前端和后端能够在当前环境下继续完成调用。

4.5. 渲染结果、轨迹输出与运行状态判断

在本次复现中，渲染结果不仅用于展示建图效果，也承担了非常重要的诊断功能。相比只观察程序是否退出，rgbdnua 图像、轨迹文件和最终 ply 模型更能反映系统当前是否处于正常工作状态。因此，实验中始终把结果输出检查作为复现流程的一部分，而不是只在最后统一查看。

具体而言，主要关注三类输出。第一类是轨迹相关结果，例如 droid_c2w 等输出文件，用于判断系统是否完成了连续位姿估计；第二类是 rgbdnua 渲染图像，用于观察真实图像、预测结果、深度、法线和不确定性等信息；第三类是最终 ply 地图模型，用于判断建图是

否完成。若程序运行结束但缺失上述任一关键结果，就说明实验链路仍存在问题，需要继续回调配置或脚本。

在实际调试中，渲染结果尤其有助于定位异常。例如，图像中的大面积黑块、空洞或局部内容消失，通常意味着当前地图状态或渲染过程存在问题；不确定性区域异常扩大，则可能反映当前建图效果不稳定。正因如此，本阶段工作并未把渲染结果仅仅视为展示图片，而是将其作为判断系统运行质量的重要依据。对于课程项目而言，这种检查方式能够保证后续实验分析建立在真实输出之上，而不是只依据日志或理论说明作出判断。

4.6. 输出组织、checkpoint 与中间结果管理

随着实验从短序列扩展到长序列，另一个逐渐突出的实际问题是输出管理。如果不加整理，长时间运行后的中间结果、模型状态和 `checkpoint` 文件不仅会占用大量存储空间，也会增加后续结果筛选和报告整理的难度。因此，本阶段还对输出组织和中间结果保留方式进行了整理。

在这部分工作中，重点关注了实验输出目录结构、关键文件保存位置以及中间结果保留策略。通过对 `checkpoint` 和结果输出进行更清晰的管理，实验结束后能够较方便地提取轨迹文件、渲染图像和最终地图模型，并据此开展后续指标整理和图像筛选。虽然这类工作不直接提升算法效果，但对于课程项目中的结果总结和报告撰写具有较强的现实意义。

4.7. 移动端专用配置的前期准备

除公开数据集外，本阶段还补充完成了移动端实时 VO 链路所需的专用配置整理。由于移动端输入不是离线图像序列，而是手机端通过网络实时发送的图像流，因此原有面向公开数据集的配置方式不能直接复用，必须单独准备适用于实时输入场景的配置入口。

在这部分工作中，建立并调整了移动端专用配置文件 `configs/mobile/mobile_vo.yaml`，使系统能够以服务端实时接收图像数据的方式运行。与离线公开数据集的 VIO 配置不同，移动端配置最终固定为 `mode: 'vo'`，即优先保证实时图像链路、前端跟踪和后端建图能够稳定工作，而不是继续强行推进视觉惯性联合运行。围绕这一目标，对输入分辨率、前端缓冲、训练迭代次数、每帧新增 Gaussian 数量以及存储管理参数进行了压缩和减负处理，以提高系统在实时联调场景下的响应能力和可运行性。

这一部分准备工作的意义在于，它将原本主要面向离线数据集的 VINGS-Mono 扩展到了实时输入场景，为后续移动端服务搭建与联调提供了配置基础。也正是在这一阶段完成专用配置后，移动端部分才具备继续开展链路搭建、问题排查和结果验证的条件。

5. 移动端实时 VO 服务复现、遇到的问题及处理

在完成离线数据集复现与系统稳定性调试之后，后续工作进一步转向移动端实时 VO 功能的构建与联调。移动端部分更强调系统链路的完整性与实时性，即手机端能否实时采集数据、服务端能否接收并处理输入，以及结果能否稳定回传至客户端显示。

5.1. 移动端目标与整体链路设计

移动端部分的核心目标，是将 VINGS-Mono 从离线数据输入模式扩展到实时输入模式，使 Android 客户端能够将采集到的图像数据发送到服务端，由服务端完成前端跟踪与

后端建图，再将结果回传到客户端显示。围绕这一目标，需要同时处理客户端、网络链路和服务端三部分问题，而不能仅依赖已有仓库中的默认脚本。

在联调初期，首先需要明确手机、本地电脑和远端服务器三者的角色分工。手机端负责图像和传感器数据采集，同时承担结果显示功能；本地电脑负责 Flutter 客户端构建、真机调试和 SSH 隧道转发；远端服务器负责运行 VINGS-Mono 后端，完成 tracking、mapping 和渲染等主要计算任务。基于这一认识，最终采用了“手机端采集 - 本地电脑中转 - 远端服务器处理”的链路结构，而不是直接让手机端连接远端服务器。

之所以采取这种方式，主要是因为手机直接连接远端服务器时，网络路径、端口映射、服务端日志检查和客户端调试会同时耦合在一起，问题定位难度较高。引入本地电脑中转之后，Flutter 真机调试、端口转发和服务端日志观察可以在同一调试链路中完成，从而显著降低部署与排查复杂度。因此，这一链路设计既是工程实现上的权宜处理，也是联调阶段提高可控性的必要选择。

5.2. 服务端运行链路搭建

在明确总体结构后，首先需要确认 VINGS-Mono 当前仓库是否真正具备移动端接入能力。为此，对仓库中与移动端相关的入口脚本和数据处理模块进行了梳理，重点检查了 `scripts/run_mobile.py`、`scripts/run_multiprocess_mobile.py`、`scripts/server/server.py` 和 `scripts/datasets/phone_server.py` 等文件。通过分析可以确认，当前仓库并非纯离线处理框架，而是已经具备了手机端 WebSocket 接收、数据重组、`tracker/mapper` 队列组织和图像回传的基础能力。

在服务端实际部署中，进一步补充了移动端专用配置文件和运行脚本，使系统能够以接收实时图像流的方式运行。例如，在 `configs/mobile/mobile_vo.yaml` 中设置了移动端专用输入模块、图像分辨率、前端参数、训练迭代次数和存储管理参数，并将运行模式固定为 `vo`。同时，增加了启动脚本和环境检查脚本，用于确认关键配置文件、运行脚本、`conda` 环境、权重文件和服务端口状态是否正常。这些准备工作使移动端实验不再依赖临时手工启动，而具备了更清晰的运行入口。

在服务端处理流程上，重点需要解决的是实时数据组织问题。与离线图像序列不同，手机端上传的是持续到达的图像帧和 IMU 数据，服务端必须完成接收、缓存、分发和回传。为此，实验中围绕图像帧进入处理队列、IMU 数据缓存、`tracker` 与 `mapper` 之间的数据传递以及回传结果的输出格式进行了调整。整体目标是让服务端能够在多进程结构下保持链路连通，即使吞吐有限，也至少能够完成连续处理和结果回传。

5.3. 客户端采集方式与参数适配

移动端部分的另一项关键工作，是将客户端从较弱的演示性采图逻辑调整为更适合实时 VO 链路的输入方式。原始客户端实现更接近“定时拍照上传”，这类方式在持续位姿估计场景中实时性不足，难以满足前端连续处理需求。因此，在联调过程中将客户端采图方式改为基于 `startImageStream(...)` 的连续视频流采集模式，并在此基础上加入最小发送间隔控制，以实现从视频流中按节奏抽帧发送。

这一修改带来的直接效果，是客户端从“离散拍照上传”转变为“连续视频流采样上传”，

从而降低采图和传输延迟,使输入形式更接近实时视觉里程计所需的连续帧序列。与此同时,为了避免客户端发送速度明显高于服务端消费速度,还保留了发送节流机制,以减轻后端队列压力。

在客户端参数适配方面,还遇到了图像分辨率与相机内参不匹配的问题。实际联调中,客户端运行时上传的分辨率并不是理论假设值,而是手机在 `ResolutionPreset.low` 下的实际输出尺寸,即 320×240 。因此,如果仍沿用原始高分辨率相机参数,前端跟踪和后端建图的几何一致性就会受到影响。针对这一问题,先对手机相机进行了棋盘格标定,获得较高分辨率下的内参,再根据实际上传分辨率对 `fu`、`fv`、`cu`、`cv` 进行缩放,最终将近似适配后的相机参数写入 `mobile_vo.yaml`。虽然这组参数不能视为严格意义上的最终标定结果,但足以支撑当前阶段的移动端 VO 联调实验。

5.4. 通信链路与时序性问题处理

在服务端与客户端都具备基本运行条件后,联调过程中的主要矛盾逐渐转向通信稳定性和整体吞吐能力。实验中,服务端日志曾频繁出现 `server2tracker_queue full`、`dropped stale frame`、`tracker waiting for server`、`mapper queue full` 等信息,对应到客户端表现则是画面更新断续、时而流畅、时而卡顿。

这一现象表明,问题并不只是网络延迟,而更像是“客户端输入速度高于服务端消费速度”导致的系统性瓶颈。进一步分析后可以发现,移动端链路的瓶颈主要来自多进程队列调度、Python 图像编解码、`tracking / mapping` 节奏不一致以及后端整体吞吐有限,而不是单纯因为 CPU 或 GPU 被完全打满。也就是说,问题更偏向流程型瓶颈,而不是某个单点资源耗尽。

针对这一问题,实验中采取了三类措施。第一,在客户端侧使用低分辨率采集并引入发送节流,主动降低输入压力;第二,在服务端保留“队列满则丢旧帧”的策略,避免旧帧无界积压;第三,对 `mobile_vo.yaml` 中部分训练和建图参数进行减负处理,使链路优先服务于当前可运行的实时 VO,而不是继续保留过重的高质量建图负载。经过这些调整,虽然系统仍不能达到十分流畅的实时效果,但整条链路已经能够在可接受速度下维持运行,并支撑“移动端实时 VO 可行”的实验结论。

5.5. 图像方向、回传布局与结果解释

除实时性外,联调中还遇到了与图像显示直接相关的多类问题。首先是图像方向不一致问题。由于手机相机流方向、服务端重组方式和客户端显示方式之间存在坐标差异,回传图像曾先后出现上下翻转、左右翻转等现象,导致显示结果与实际观察方向不一致。针对这一问题,在 `phone_server.py` 中对运行时图像方向进行了适配,并通过真机多轮验证确认最终显示方向与实际观察一致。

第二类问题是回传图像拼接布局不稳定。实验中曾出现 `cv2.resize ... !ssize.empty()`、`np.concatenate ... dimensions must match` 等错误,说明原有辅助面板拼接逻辑在移动端场景下不够稳健。原因在于 `mapper.vis_rgbdnua` 的输出尺寸和内容并不总是固定,而原有拼接方式对宽高一致性要求较高,一旦出现空图或尺寸变化,就容易报错。针对这一问题,对 `run_multiprocess_mobile.py` 中的回传布局进行了多轮调整,先后尝试过自适应拼接、只返回

主图、恢复简单拼接等方案，最终收束为较稳定的“两行布局”：第一行显示主渲染图，第二行显示辅助视图面板。

第三类问题是结果语义解释问题。联调初期，客户端界面上的图像一度被误认为是手机原始 RGB 预览，导致对黑块、空洞和异常区域的理解出现偏差。后续在梳理服务端回传逻辑后，确认客户端显示的是服务器回传的拼接可视化结果，而不是手机原始相机画面。其中，上排为服务端主渲染图，下排为辅助视图面板。基于这一认识，又在客户端界面中加入了简短说明，使结果展示更易理解。这样一来，后续对局部黑块或内容缺失的分析就能够归因到服务端渲染与地图状态，而不是误判为手机采集问题。

5.6. 连接管理与客户端稳定性改进

在联调过程中，还观察到服务端日志中会在运行中途多次再次出现 `Client Connected!`，说明客户端存在重复建连现象。这一问题虽然不属于 SLAM 算法本身，但会影响链路稳定性判断，也容易在调试时引入额外干扰。

针对这一问题，进一步检查了移动端客户端的 `WebSocket` 管理逻辑，并在客户端中增加了连接中的状态保护、连接前检查、重复点击禁用、重连前主动关闭旧连接以及页面销毁时自动断开连接等机制。通过这些处理，重复建连现象得到控制，服务端日志中不再无故频繁出现新的连接建立事件，整条链路的可控性也有所提升。

这一修改说明，在移动端联调中，问题并不全部来自后端算法或网络本身，客户端的状态管理同样会显著影响实验效果。对于课程项目来说，这类问题虽然不如 VIO 失败或长序列崩溃那样显眼，但同样是实际工程联调中必须处理的部分。

5.7. 为什么最终固定为 VO 模式而未继续推进 VIO

在移动端部分，最关键也最需要明确说明的结论，是当前实验最终固定为 VO 模式，而没有继续推进实时 VIO。需要指出的是，这一结果并不是因为项目完全不支持 IMU，也不是因为移动端链路中完全没有传感器输入，而是在实际联调中，VO 链路已经能够基本跑通，而 VIO 在当前条件下无法稳定工作。

从链路上看，移动端客户端在实验过程中并非只发送图像，IMU 数据同样能够传输到服务端；服务端侧也保留了 IMU 接收与缓存逻辑。因此，问题并不在于“有没有 IMU”，而在于“IMU 是否能够稳定参与视觉惯性初始化与后续估计”。在实际运行中，一旦系统进入视觉惯性初始化阶段，就容易在 `init_VI()`、`VisualIMUAlignment()` 等步骤附近报错，典型现象包括 `Found zero norm quaternions in quat`、激励不足以及初始化后立即崩溃等。这说明当前移动端链路下，视觉和惯性信息尚未形成足够稳定的一致性。

进一步分析后可以看出，这一问题并不是单一参数设置错误，而是由多方面因素叠加导致的。首先，手机端图像与 IMU 的时间同步精度无法与标准公开数据集完全相比；其次，手机相机与 IMU 的外参并未进行严格标定；再次，手机 IMU 的噪声特性与公开数据集中的惯性数据存在差异；最后，当前仓库中的移动端相关实现更偏向实验性链路，对实时 VIO 的适配并不充分。也就是说，即使链路表面上“具备 IMU 输入”，也不意味着能够直接稳定实现视觉惯性联合估计。

针对这一问题，实验中并非没有尝试修复，而是进行过若干方向的调整，例如提高 I

MU 激励阈值、回退部分前端改动、讨论跳过视觉惯性初始化，以及让服务端继续接收 I MU 但不真正将其用于后端计算等。然而继续沿着 VIO 方向修补，会显著增加前端状态结构和整体调试复杂度，并有可能破坏已经基本跑通的 VO 链路稳定性。因此，在课程项目时间和任务边界下，最终选择不再继续强行推进移动端 VIO，而是将实验目标收束为“保证实时 VO 链路稳定可运行”。

因此，本阶段关于移动端的客观结论应当是：移动端实时 VO 已经完成联调验证，能够实现手机图像采集、服务端处理与结果回传；而移动端 VIO 虽然在代码结构上具备一定基础，但受限于时间同步、外参标定、IMU 噪声特性和代码适配，当前仍无法稳定运行，因此被明确保留为后续问题，而不是在本次课程实验中强行宣称完成。

6. 实验结果与分析

在完成公开数据集复现、系统调试与移动端联调之后，可以从离线数据集实验和移动端实时实验两个方面对结果进行总结。实验结果的评价标准并不单纯以“是否完全达到论文精度”为唯一依据，而更关注以下几个方面：其一，系统是否能够完成完整运行；其二，是否能够输出可分析的轨迹、渲染和地图结果；其三，在长序列和实时场景下，系统是否具备基本稳定性。

为更清晰地比较不同序列上的复现情况，进一步对各实验序列的轨迹指标、渲染指标和有效输出帧数进行了汇总，如下表所示。需要说明的是，该表主要用于反映系统在不同场景下的运行完成度、结果质量和可分析性。其中，部分长序列虽然能够完整输出结果，但轨迹误差仍然较大，因此相关结果更适合作为复现与调试阶段的实验记录，而不宜直接作为论文精度复现结论。

序列	ATERMSE	tREL	rREL	PSNR	SSIM	LPIPS	Renderframes	结论
Hotel	N/A	N/A	N/A	26.971	0.802	0.305	114	缺少轨迹评估，主要用于渲染结果展示
Small city	4.025 m	3.66%	1.018 deg/100m	20.210	0.694	0.409	305	整体运行最稳定，可作为代表性复现结果
KITTI 0027	55.349 m	72.76%	56.305 deg/100m	15.889	0.566	0.460	238	完成运行，但轨迹与渲染结果均不理想
KITTI 0028 try	239.742 m	67.81%	37.369 deg/100m	15.021	0.441	0.627	1467	完成长序列运行，但轨迹误差较大
KITTI360 0000 lite	158.926 m	77.53%	43.507 deg/100m	28.054	0.825	0.189	699	图像重建效果较好，但轨迹误差较大
KITTI360 0010 try	515.565 m	140.86%	33.192 deg/100m	26.690*	0.819	0.183	976	长序列可运行，渲染结果较好，但定位精度不足

表 1 主要实验结果指标汇总

注：KITTI360 0010 try 的 PSNR 为对有效帧统计后的结果，个别帧存在异常值，因

此该指标主要用于参考，不宜孤立解读。KITTI360 0000 lite 是因为完整的序列总是会出现超出显存的问题，所以采用了截取部分子序列运行的方式。

6.1. 离线数据集复现结果

从表中可以看出，不同数据集上的复现结果存在明显差异。其中，smallcity 序列在当前实验中表现最为稳定，其 ATE、tREL 和 rREL 均明显优于多数长序列结果，同时也能够输出较完整的渲染图像和地图结果。因此，从“轨迹稳定性”和“结果完整性”两方面综合考虑，smallcity 可以视为本次复现中完成度较高的一组代表性结果。

图 1 展示了 smallcity 序列中的一帧 rgbdnua 可视化结果。图中不仅包含真实输入图像和系统渲染结果，还同时给出了深度、法线和不确定性等辅助信息。从可视化效果来看，系统已经能够在道路场景中恢复出较完整的地面结构与前方区域，渲染结果与真实图像在整体布局上基本一致。这说明在中等规模场景下，VINGS-Mono 已经具备较好的运行稳定性与结果可解释性。

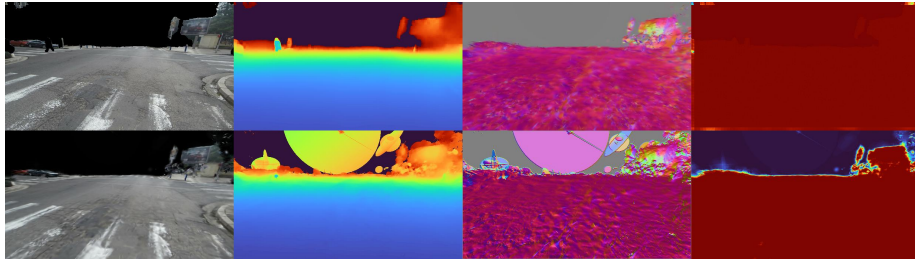


图 1 smallcity 序列渲染与辅助可视化结果

相较之下，KITTI 数据集更能反映系统在真实驾驶场景中的复现情况。表 1 中可以看到，KITTI 0027 和 KITTI 0028 序列均已完成运行，并输出了轨迹和渲染结果，但从 ATE、tREL、rREL 以及渲染指标来看，其整体结果并不理想。其中，KITTI 0028 try 虽然完成了较长序列运行，并输出了 1467 帧渲染结果，但轨迹误差仍然较大，说明系统已经具备可运行性，但在当前实验条件下，驾驶场景中的参数适配和稳定性仍有进一步优化空间。

图 2 给出了 KITTI 0028 序列中的一帧 rgbdnua 结果。可以看到，系统已经能够在道路和车辆场景中完成基本建图与渲染，真实图像和系统输出在地面区域、前方结构和道路边界上具有一定对应关系。但与此同时，也可以观察到渲染图中仍存在黑色缺失区域、局部细节不够稳定等问题。这与表 1 中较高的轨迹误差和较弱的渲染指标是一致的，说明该序列更适合作为“完成复现并得到结果输出”的证据，而不宜直接作为高质量定位与重建结果来使用。

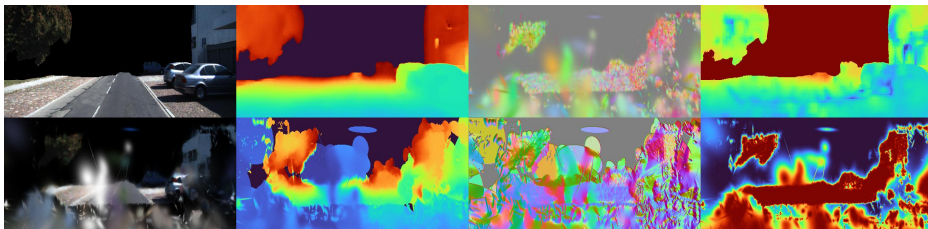


图 2 KITTI 0028 序列渲染与辅助可视化结果

从结果属性上看，KITTI 部分的意义主要体现在两个方面：第一，公开数据集复现链

路已经建立，系统能够在驾驶场景中完成从输入到输出的完整流程；第二，虽然结果可用，但轨迹和渲染质量仍有不足，这表明当前实验更适合支撑“复现完成与系统可运行”的结论，而不是“精度完全达到论文水平”的结论。

6.2. 长序列与大场景结果分析

KITTI360 相关实验更能体现系统在长序列和大场景条件下的运行特性。与 KITTI 相比，KITTI360 不仅场景尺度更大，而且对显存、训练稳定性和结果输出完整性提出了更高要求。因此，这部分实验的价值不仅体现在最终指标上，也体现在系统是否能够在有限资源条件下完整跑通。

从表 1 可以看出，KITTI360 0000 lite 和 KITTI360 0010 try 在渲染指标上并不差，例如 PSNR、SSIM 和 LPIPS 均优于部分 KITTI 结果，这说明系统在长序列中仍然能够持续输出图像重建结果。但与此同时，这两组序列的轨迹误差都明显较大，说明当前复现结果呈现出“图像重建效果尚可，但定位精度不足”的特点。换言之，这部分结果更适合用于说明系统在长序列上的可运行性，而不是用于证明其高精度定位表现。

图 3 展示了 KITTI360 0010 try 序列中的一帧输出结果。与前面的 smallcity 和 KITTI 0028 相比，这张图中的局部块状失真、渲染不连续和内容退化现象更加明显。这类图像不适合作为“最佳效果展示图”，但它能够真实反映长序列实验中的困难：即使系统已经能够完整运行并保留结果，长时间运行后渲染质量和局部稳定性仍可能受到较大影响。

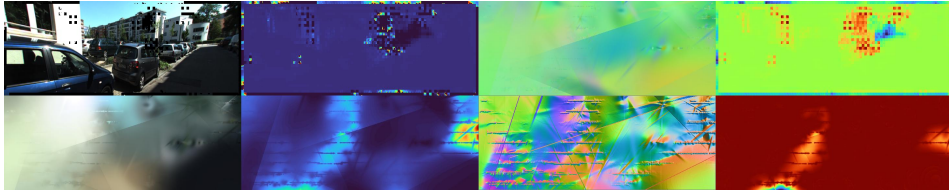


图 3 KITTI360 0010 try 序列输出结果

此外，Hotel 序列在本次实验中的作用也与 KITTI、KITTI360 略有不同。由于该序列缺少适合直接对照的轨迹评估，因此它更适合用于展示系统在室内场景中的渲染输出能力，而不适合与其他序列一起做轨迹精度比较。从表 1 可以看出，Hotel 的渲染指标相对较好，这说明在部分场景中，系统能够获得较稳定的图像重建结果，但由于缺少轨迹评价，相关结果更适合作为辅助展示而非核心结论。

综合来看，KITTI360 和 Hotel 这类结果共同说明：VINGS-Mono 的复现结果不能只从单一指标出发进行判断。在大场景长序列条件下，系统可能表现为“能够完整运行并生成较多渲染结果，但轨迹误差仍较大”；而在室内序列上，则可能表现为“渲染结果较好，但缺少完整轨迹评价”。因此，对本次实验结果的理解需要同时结合轨迹指标、渲染指标和运行完成度，而不能仅凭某一项单独下结论。

6.3. 移动端实时 VO 联调结果

与离线实验不同，移动端部分的核心结果并不主要表现为某个单独的数值指标，而在于整条实时链路是否能够实际跑通。围绕这一目标，实验最终完成了手机端图像采集、数据传输、服务端处理以及结果回传显示的完整流程。也就是说，移动端实验已经从“理论支持移

动端接入”推进到“完成实时联调验证”的阶段。

图 4 展示了移动端联调过程中的一帧回传结果。需要说明的是，客户端界面中显示的并不是手机原始相机预览，而是服务端返回的可视化拼接图，其中包括主渲染结果和辅助视图面板。从图 4 中可以看到，系统已经能够接收手机端输入并回传处理后的图像结果，这就是移动端链路跑通的重要证据。虽然其显示效果仍然偏向调试用途，局部区域也存在不稳定现象，但它已经验证了“手机端输入 - 服务端处理 - 客户端显示”这一链路是可以工作的。

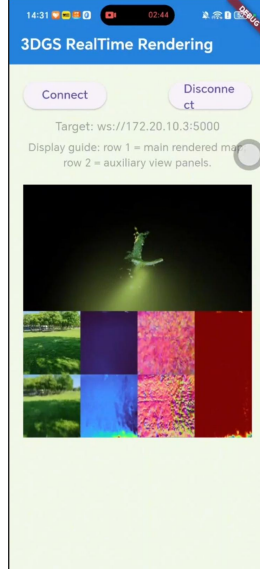


图 4 移动端实时 VO 联调回传结果

从联调现象来看，移动端部分已经取得了两个比较明确的结果。第一，客户端与服务端之间的通信链路已经打通，手机端能够采集图像并通过 WebSocket 发送到服务端，服务端可以完成相应处理并将结果回传。第二，在当前条件下，系统能够较稳定支持实时 VO，而不是仅停留在离线演示层面。这说明 VINGS-Mono 在经过适配后，已经能够在一定程度上支持面向移动端的实时运行验证。

但与此同时，移动端结果也明确暴露出系统当前的边界。首先，联调过程中仍存在明显卡顿和吞吐不足现象，说明后端 tracking / mapping 链路尚未达到理想的实时处理能力。其次，客户端回传结果更多是面向调试的渲染与辅助视图拼接图，而不是高质量最终渲染，因此其主要价值在于验证链路，而不是展示视觉效果。最后，虽然客户端同时上传了 IMU 数据，但当前移动端 VIO 仍无法稳定工作，因此本次移动端实验最终固定为 VO 模式。换言之，移动端部分的真实结论应当是：实时 VO 已经跑通，实时 VIO 暂未完成。

6.4. 综合分析

首先，从实验完成度来看，本次工作完成了多组公开数据集的实际运行、结果输出与移动端实时联调验证。无论是 smallcity、KITTI、KITTI360 的离线实验，还是移动端实时 VO 的联调结果，都说明系统已经被落实为可执行、可观察、可分析的实验流程。

其次，从结果性质来看，当前最强的结论并不是“所有精度指标都达到论文水平”，而是“复现链路已经建立，并能够在不同场景下输出真实结果”。其中，smallcity 更适合作为本次复现中相对稳定的一组代表性结果；KITTI 结果说明系统已经能够在驾驶场景中完成完整

运行，但结果质量仍有不足；KITTI360 结果则进一步表明，大场景长序列的复现重点已经从“是否能跑”转向“是否能稳定跑完并保留结果”；移动端部分则说明实时 VO 联调已经具备基本可行性。

最后，从系统局限来看，本次实验结果也如实反映出两个主要限制：其一，长序列条件下的轨迹精度和渲染稳定性仍不理想；其二，移动端 VIO 在当前链路下尚未稳定实现。

7. 总结

本次课程项目以 VINGS-Mono 为基础展开，个人负责内容主要集中在两个方面：一是 VO / VIO 相关公开数据集的复现、配置调整与运行调试；二是移动端实时 VO 功能的构建与联调。

在公开数据集复现部分，本次工作完成了 KITTI、KITTI360 等序列的运行验证，并对不同场景下的结果进行了整理与分析。实验表明，VINGS-Mono 已经能够在多个场景中完成从输入、跟踪、建图到渲染输出的基本流程，其中部分中等规模序列能够得到相对稳定的结果，而长序列和大场景实验虽然可以跑通，但在轨迹精度、渲染稳定性和资源控制方面仍存在明显挑战。在移动端部分，本次工作完成了从客户端采集、网络传输、服务端处理到结果回传显示的实时链路联调，验证了 VINGS-Mono 在移动端实时输入场景下的基本可行性。实验结果表明，当前系统已经能够较稳定地支持移动端实时 VO，但在视觉惯性初始化、时间同步、外参标定和 IMU 适配方面，移动端 VIO 无法稳定运行，仍需进一步研究和调试。

总体来看，本次课程项目的个人工作重点在于面向真实实验条件完成工程复现、系统调试和联调验证。从最终结果看，公开数据集复现链路和移动端实时 VO 链路均已建立，说明论文方法和开源项目已经在当前实验环境下具备了较强的可执行性。与此同时，实验中暴露出的长序列稳定性问题和移动端 VIO 问题也表明，从“能够运行”到“高质量稳定运行”之间仍存在一定差距。对本人而言，本次课程项目不仅加深了对视觉 SLAM 与 Gaussian Splatting 系统实现方式的理解，也进一步体会到工程复现、参数适配与系统联调在实际研究和开发工作中的重要性。